Bootstrapping Vision-Language-Action Models for New Robot Embodiments with Staged Reinforcement Learning

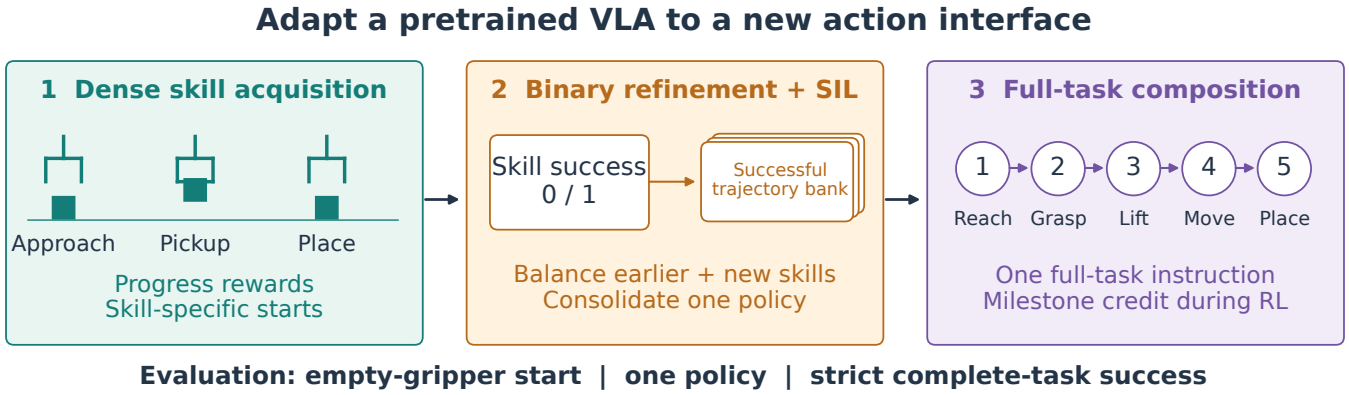


Fig. 1. Proposed pipeline. Dense feedback acquires skills; binary refinement and balanced self-imitation consolidate them; milestone-based RL learns the complete task under one instruction. Skill-specific resets are used during acquisition. Full-task evaluation starts with an empty gripper and uses no external switching between specialists.

WORKING DRAFT — 28 October 2026

***Abstract*—A pretrained vision-language-action (VLA) model can produce few successful rollouts when transferred to a new action interface. Sparse rewards then hinder learning, while sequential skill training can erase earlier behavior. We propose a pipeline of dense skill acquisition, binary refinement with balanced self-imitation, and milestone-based full-task composition. Intermediate feedback guides exploration, successful experience consolidates skills, and milestone credit connects them under one instruction. We implement the pipeline with a SmolVLA residual controller and LoRA, and examine reward design with OpenVLA-OFT on LIBERO-Spatial. The controller achieves 61.43% strict pick-and-place success on a separate CDPR development evaluation; a later continuation logs 70.61% on the in-run validation panel, pending separate evaluation. Balanced self-imitation raises reaching success from 8.0% to 31.1% while preserving placement. On LIBERO-Spatial, dense feedback reaches 65.3% success versus 61.7% with terminal-only reward at the same update budget.**

***Index Terms*—Robot learning, reinforcement learning, manipulation, vision-language-action models, robot adaptation.**

I. Introduction

Vision-language-action (VLA) models provide useful visual and language representations for robot control. Transferring these representations to a new robot still requires learning how its actions affect the scene. If the initial policy rarely completes the task, binary success provides little guidance for reinforcement learning.

Consider *put the apple on the plate*. A failed rollout may contain a correct approach and grasp followed by a poor release. Treating the entire attempt as a failure discards this distinction. Training the parts separately helps exploration, but a policy can forget earlier skills while learning later ones. It must then connect the retained skills into a complete behavior.

We address these problems through the pipeline in Fig. 1. Dense rewards first guide skill acquisition. Binary refinement then targets completion, while balanced self-imitation retains successful behavior across instructions. Finally, milestone-based RL trains one policy to execute the full task from an empty-gripper start.

The pipeline separates feedback design and skill consolidation from the robot-specific action mapping. We implement it with SmolVLA, LoRA, and a residual action controller. Experiments examine complete-task learning and skill retention in cable-driven parallel robot (CDPR) simulation, and intermediate feedback with OpenVLA-OFT on LIBERO-Spatial. An ongoing WX200 study extends the evaluation to a five-DoF serial manipulator.

Our contributions are a staged adaptation pipeline, a practical mechanism for retaining and composing skills, and empirical results on forgetting, recovery, and reward design. The LIBERO study evaluates the feedback component; it does not test the full adaptation schedule.

II. Related Work

VLA adaptation. OpenVLA [1], SmolVLA [2], and OpenVLA-OFT [3] provide pretrained policies and adaptation methods. We retain a pretrained action expert and learn a residual for the target interface. Our LIBERO study instead fine-tunes an OpenVLA-OFT-based policy through discrete action-token likelihoods.

Residual learning and consolidation. Probe, Learn, Distill combines residual RL, autonomous data collection, and distillation [4]. We build on these ingredients to link skill acquisition, retention, and full-task composition. Self-imitation here means fitting successful policy-generated experience, including trajectories retained from earlier training.

Feedback for reinforcement learning. SimpleVLA-RL applies group-relative optimization to VLA rollouts [5], following GRPO [6]. Binary rewards provide no within-group contrast when all rollouts share the same outcome [7], [8]. Intermediate feedback can distinguish partial progress. Unlike potential-based shaping [9], our task-specific rewards do not carry a policy-invariance guarantee.

III. Methods

The goal is to adapt a pretrained VLA to complete language-conditioned manipulation tasks on a target robot. The policy receives camera images, proprioception, and an instruction, predicts an action chunk, and executes a prefix before observing again. We evaluate complete-task success independently of the reward used for training.

During acquisition, each skill has its own instruction, success condition, and starting states. For example, placement can begin with an object already held. During composition and final evaluation, the robot starts with an empty gripper and receives one complete-task instruction. This distinction prevents isolated skill performance from being treated as full-task success.

A. Dense Acquisition, Binary Refinement, and Self-Imitation

We first train approach, pickup, and placement skills with dense rewards. Distance reduction and manipulation events guide exploration before skill success becomes common. After dense acquisition, we refine the skills using binary success rewards.

Successful trajectories are retained by instruction. When later training degrades earlier skills, self-imitation fits a balanced mixture of earlier and current successes:

$$\mathcal{L}_{\mathrm{SI}} = \mathbb{E}_{\mathrm{success}} \left[\|\hat{a} - a\|^2 \right]. \tag{1}$$

Here $\hat{a}$ is the predicted action and a is the recorded executed action. The average balances instructions and excludes unused action slots. Policy-dependent features are refreshed before fitting. Balancing matters because imitating only the newest skill can reinforce forgetting.

Algorithm 1 summarizes the schedule. Progress is checked with fixed skill evaluations. After an initial training period, automated stages can end at a success target or a plateau; budget exhaustion also ends a stage. The CDPR sequence used validation-guided manual transitions. After consolidation, all learned skills are evaluated again. Automated runs retain the consolidation update only when recovery improves the degraded skills on average and keeps other skills within tolerance.

Algorithm 1 Staged VLA adaptation

Require: Pretrained VLA, skill instructions, fixed evaluations
Initialize the target action controller and success banks
for reward mode: dense acquisition, then binary refinement **do**
  for skill: approach, pickup, placement **do**
    repeat
      Collect same-scene rollout groups for the skill
      Update with $\mathcal{L}_{\mathrm{RL}}$; retain successful trajectories
      Evaluate skill success
    until target, plateau, or stage budget ends training
    Evaluate retention of all learned skills
    if earlier skills have degraded **then**
      Balance earlier and current successful experience
      Fit $\mathcal{L}_{\mathrm{SI}}$; validate recovery and retention
    end if
  end for
end for
repeat
  Collect full-task rollouts from empty-gripper starts
  Assign milestone credit; protect achieved prefix skills
  Update with $\mathcal{L}_{\mathrm{RL}}$; evaluate complete-task success
until target, plateau, or full-task budget ends training
return One policy for the complete instruction

B. SmolVLA with an Embodiment-Specific Residual

Figure 2 shows one implementation of the pipeline. The pretrained SmolVLA expert supplies an action prior, mapped to the target control interface. In our five-channel controller, eight action slots give

$$p \in \mathbb{R}^{40} \qquad (8 \text{ slots} \times 5 \text{ channels}). \tag{2}$$

A frozen visual projection supplies 512 features; six proprioceptive values complete the 518-dimensional observation. The residual receives this observation and the 40-dimensional prior, and predicts a bounded correction. The combined action is bounded before execution.

LoRA adapts the action expert during earlier training. In the evaluated full-task continuation, the inherited VLA and LoRA weights are frozen while the residual and exploration parameters are trained. The backbone, action mapping, and feature dimensions can change with the embodiment; the training schedule remains the same.

C. Milestone-Based Full-Task Composition

After skill consolidation, one policy learns the complete instruction from empty-gripper starts. No external controller switches between specialist policies. Approach, held pickup, and final placement serve as milestones for assigning credit within a rollout.

The main rule is simple: a later placement failure should not remove the local credit for a successful approach or pickup. We compare milestone outcomes within groups of rollouts from the same scene. Actions receive credit for the milestone they contribute to, while full-task success provides an additional preference for completing the chain. A negative completion bonus is suppressed on already achieved approach and pickup milestones. Placement receives its completion credit once (Fig. 3).

This rule preserves useful partial behavior while training the remaining task. It controls the assigned credit; it does not prevent interference through shared network parameters.

D. Stabilized Policy Updates

We use a clipped group-relative policy loss with entropy regularization:

$$\mathcal{L}_{\mathrm{RL}} = \mathcal{L}_{\mathrm{clip}} - \eta \mathcal{H}. \tag{3}$$

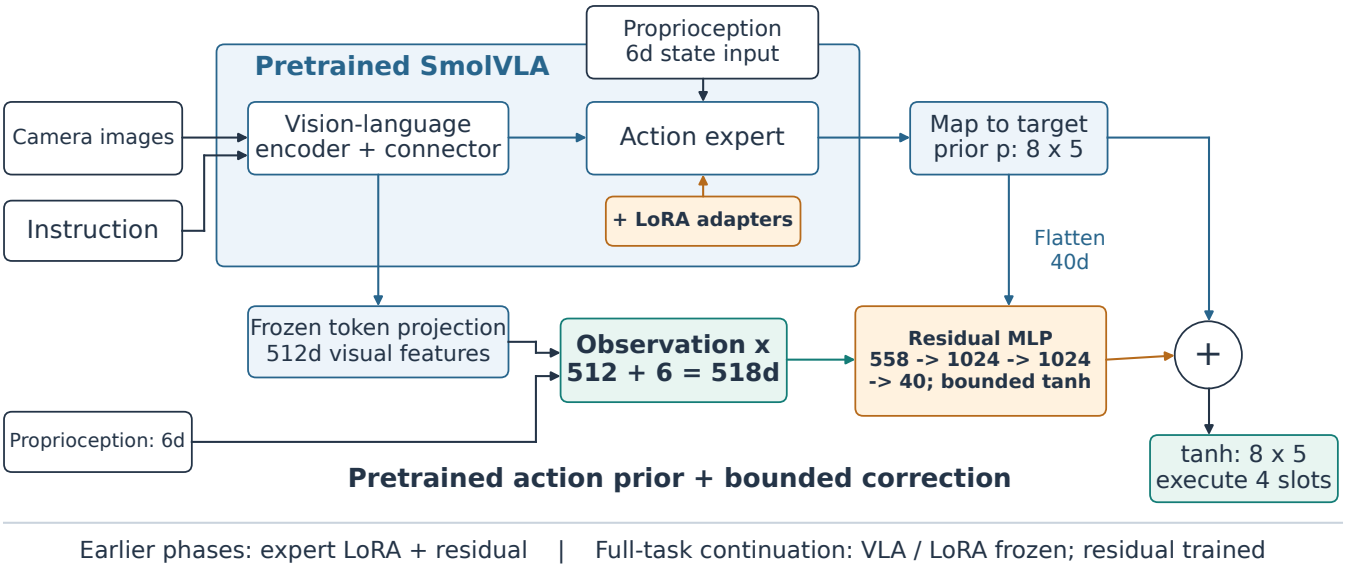


Fig. 2. SmolVLA implementation of the pipeline. The added residual combines 518 observation features with a 40-dimensional action prior. LoRA adapts the pretrained expert during earlier training; the evaluated full-task continuation freezes the VLA and LoRA weights. Feature and action dimensions refer to the five-channel controller.

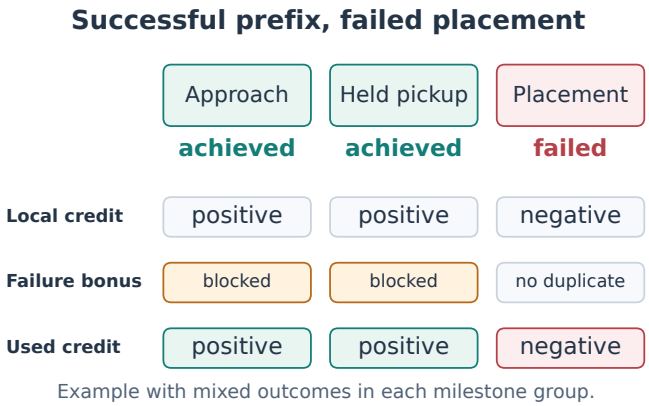


Fig. 3. Credit assignment when approach and pickup succeed but placement fails. Local credit remains available for the achieved skills. The negative completion bonus is blocked on these skills, while placement is trained from its own outcome. The diagram assumes groups with informative milestone comparisons.

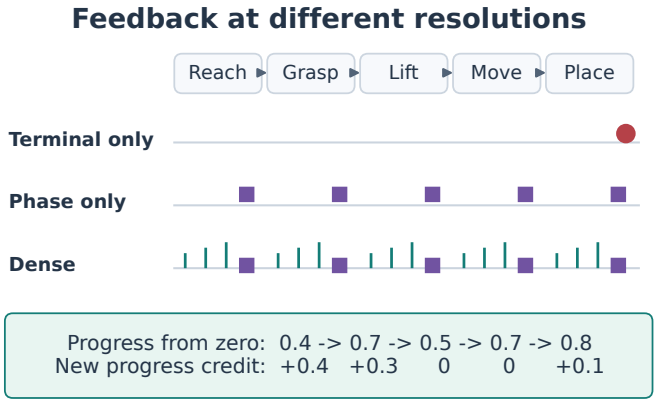


Fig. 4. Feedback resolution. Terminal-only reward distinguishes final success. Phase rewards mark subgoals, and dense feedback also credits new progress within each phase. Revisiting an earlier maximum gives no additional progress reward.

The clipped loss compares rollouts within each scene group and limits the incentive for large changes in action likelihood. We use separate upper and lower clipping bounds. The entropy term $\mathcal{H}$, weighted by η, maintains exploration during manipulation and composition. Together, these choices help keep early updates conservative while successful behavior is still rare.

For bounded continuous actions, likelihoods are computed from the sampled Gaussian commands before clipping, while the robot executes the clipped commands. This keeps the loss consistent with the sampling process at action limits [10]. Self-imitation uses the simpler regression loss in Eq. (1).

E. Intermediate Feedback and Rollout Groups

Feedback can resolve progress at different levels (Fig. 4). In the LIBERO implementation, a tracker follows reach, grasp, lift, move, and place. Phase rewards mark completed subgoals. Dense rewards also credit progress within the active phase, but only when it exceeds the best value reached so far. Returning to the same position earns no repeated progress reward. The progress baseline resets when the next phase begins.

Shaping is bounded and added to the terminal reward; the task's success verifier remains unchanged. Optional group filtering selects groups by their fraction of terminal successes, separately from dense reward construction.

The motivation follows from a simple group calculation. For G independent rollouts from one scene, each with success probability p, the probability of observing both success and

failure is

$$P_{\text{mixed}} = 1 - p^G - (1-p)^G. \quad (4)$$

With eight rollouts, this probability is only 21.6% at 3% success, but 94.2% at 30% success. Milestones can therefore provide informative comparisons before complete-task success becomes common. Dense feedback can also distinguish all-failure rollouts. A filter that removes such groups discards this potential signal.

IV. Experiments

We evaluate three aspects of the approach: full-task composition, retention during sequential learning, and intermediate feedback. SmolVLA experiments test the adaptation pipeline in CDPR simulation. OpenVLA-OFT experiments test reward design on the ten LIBERO-Spatial tasks. The ongoing WX200 study examines another embodiment.

A. SmolVLA: Complete-Task Adaptation

The CDPR simulation uses MuJoCo Warp/MJ-Lab, four object categories (apple, orange, potato, and tomato), and plate or bowl destinations. The action interface controls Cartesian translation, yaw, and the gripper. Strict success requires grasp, held lift, intentional release, and correct placement, without carry slip or wrong-placement events.

Separate evaluation covers 512 development scenes with four repeats each, excluding the in-run validation panel. Each attempt starts with an empty gripper and uses one full-task instruction, without teacher assistance. A later continuation supplies only TensorBoard in-run validation, with 1,024 attempts per reading. These two protocols are reported separately. All attempts, including numerical failures, remain in their respective denominators. The separately evaluated controller completes 1,258 of 2,048 attempts (61.43%; Table I). Held lift occurs in 85.89% of attempts, but only 71.52% of held-lift attempts reach strict completion. Transport and release remain the main bottleneck. Completion rates are 68.58% for plates and 54.22% for bowls.

A subsequent continuation from 40.06M to 60.03M selected actions logs 723/1,024 strict successes (70.61%) at its endpoint on the in-run validation panel. Plate and bowl success are 393/512 (76.76%) and 330/512 (64.45%). The panel peak is 728/1,024 (71.09%), selected from 81 readings; the final 20 readings average 69.13%. These correlated measurements were inspected during training. Separate evaluation of the new endpoint is unavailable, so they establish neither a matched gain over 61.43% nor a new checkpoint promotion.

B. Skill Forgetting and Recovery

Sequential training strongly affects earlier instructions (Fig. 5). Reaching success falls from a mean of 64.1% to 5.3% after placement RL, and to 1.8% after placement-only imitation.

In a separate retention experiment, balanced self-imitation raises reaching from 8.0% to 31.1%. Plate placement changes from 50.0% to 52.3%, and bowl placement from 23.05% to 27.65%. The bank combines earlier reaching successes with current placement successes. Each skill is evaluated under its own fixed reset protocol; placement skill tests can begin with a held object.

Recovery is partial, but it preserves placement while improving reaching. Placement-only imitation does not provide this balance. These skill-level results do not isolate the contribution of self-imitation to final full-task success.

TABLE I

CDPR complete-task results under distinct protocols. Separate evaluation: 512 scenes, four repeats (2,048 attempts). Later in-run endpoint: 1,024 attempts, pending separate evaluation. Each rate uses its full protocol denominator; event flags can overlap.

Outcome	Attempts (%)
Separate development evaluation (40M)	
Physical grasp	89.65
Held lift	85.89
Strict completion (1,258/2,048)	61.43
Carry slip	23.83
Wrong placement	19.63
Numerical failure	1.37
Later in-run validation endpoint (60M)	
Physical grasp	92.48
Held lift	88.28
Strict completion (723/1,024)	70.61
Carry slip	16.70
Wrong placement	14.16
Numerical failure	0.88

C. OpenVLA-OFT: Intermediate Feedback on LIBERO

We fine-tune an OpenVLA-OFT-based policy using SimpleVLA-RL and LoRA on LIBERO-Spatial [11]. This benchmark uses a conventional serial-arm interface, allowing reward design to be studied separately from learning a new action mapping.

Aggregate runs share a 61.0% initialization. Each aggregate endpoint uses 300 validation rollouts and the native goal verifier. The primary dense and terminal-only conditions use the same terminal reward weight and update budget. Additional conditions examine mixed-outcome group filtering and phase-only feedback. Table II states the update budgets and terminal reward weights.

A separate low-success task starts at 2.0%, measured over 1,000 rollouts. Both conditions in that comparison use the same terminal-outcome group filter. At 160 updates, dense feedback reaches 65.3% success versus 61.7% with terminal-only reward, a difference of 3.6 percentage points. Filtered dense feedback reaches 64.7% at 50 updates.

With the lower terminal reward weight, phase-only and dense feedback reach 62.3% and 61.0%, respectively. Their phase weights also differ, so this comparison does not isolate within-phase progress. On the low-success task, dense and terminal-only training reach 18% and 11% after 20 updates, from the same 2% initialization.

The matched-budget comparisons support intermediate feedback in these settings. Different budgets across the other conditions prevent a general efficiency claim.

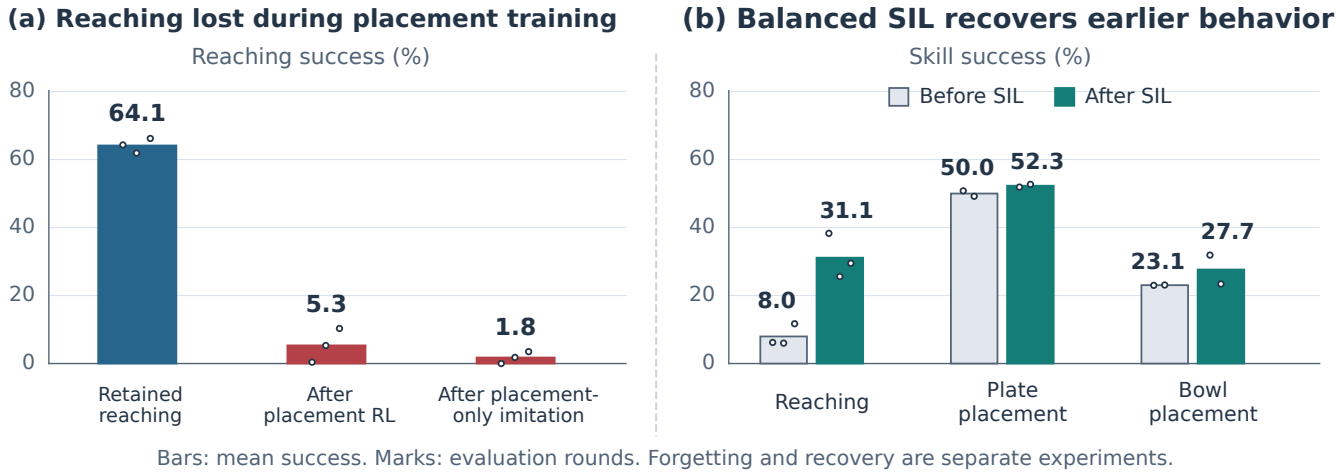


Fig. 5. Forgetting and recovery across instructions. Left: reaching after placement RL and placement-only imitation, with three evaluation rounds per condition. Right: before and after balanced self-imitation, with three reaching rounds and two placement rounds. Bars show means; marks show individual rounds. The panels are separate experiments.

TABLE II
LIBERO REWARD COMPARISON. λ IS THE TERMINAL REWARD WEIGHT; SUCCESS AND Δ ARE PERCENTAGES AND PERCENTAGE-POINT GAINS. AGGREGATE ENDPOINTS USE 300 VALIDATION ROLLOUTS. UPDATE BUDGETS ARE SHOWN FOR EACH CONDITION.

Configuration	Updates	Success	Δ
LIBERO-Spatial: initial 61.0%			
Terminal only, $\lambda = 5$	160	61.7	+0.7
Dense, $\lambda = 5$	160	65.3	+4.3
Filtered dense, $\lambda = 5$	50	64.7	+3.7
Dense, $\lambda = 1$	110	61.0	+0.0
Phase only, $\lambda = 1$	110	62.3	+1.3
Low-success task: initial 2.0% (1,000 rollouts)			
Terminal only	20	11.0	+9.0
Dense	20	18.0	+16.0

D. Additional Embodiment: WX200

An ongoing experiment applies the pipeline to the five-DoF WX200 serial manipulator. This setting changes the robot mechanism while retaining the skill-learning and composition schedule. Figure 6 reserves the setup and rollout illustrations. Quantitative success rates will be added after the evaluation is complete.

E. Scope of the Evidence

The complete-task results come from one SmolVLA training lineage; evaluation repeats are not independent training seeds. The 60M endpoint has only in-run panel measurements, with no new scene-level evaluation records or checkpoint hash supplied. Repeated panel inspection and peak selection limit generalization claims; the 40M controller remains the latest separately evaluated development result. The retention results summarize a small number of rounds, and the LIBERO results are descriptive endpoint comparisons. Neither study isolates every stage of the pipeline. Earlier assisted training is part of the inherited CDPR policy; the reported full-task evaluation uses no assistance.

V. CONCLUSION

We presented a VLA adaptation pipeline that acquires skills with dense feedback, refines and consolidates them, and learns complete-task execution through milestone credit. SmolVLA experiments show full-task learning and partial recovery of forgotten skills. OpenVLA-OFT experiments support intermediate feedback on LIBERO-Spatial. Together, these results motivate treating exploration, retention, and composition as connected parts of adaptation.

VI. FUTURE WORK

We will complete the WX200 evaluation to test the pipeline on another embodiment. Further experiments will compare training stages under matched budgets, repeat training across independent seeds, and evaluate unseen scenes. Automatic phase detection and progress estimation could also reduce the need for manually designed reward predicates.

REFERENCES

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09246

[2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, "SmolVLA: A vision-language-action model for affordable and efficient robotics," *arXiv preprint arXiv:2506.01844*, 2025. [Online]. Available: https://arxiv.org/abs/2506.01844

[3] M. J. Kim, C. Finn, and P. Liang, "Fine-tuning vision-language-action models: Optimizing speed and success," *arXiv preprint arXiv:2502.19645*, 2025. [Online]. Available: https://arxiv.org/abs/2502.19645

[4] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu, "Self-improving vision-language-action models with data generation via residual RL," *arXiv preprint arXiv:2511.00091*, 2025. [Online]. Available: https://arxiv.org/abs/2511.00091

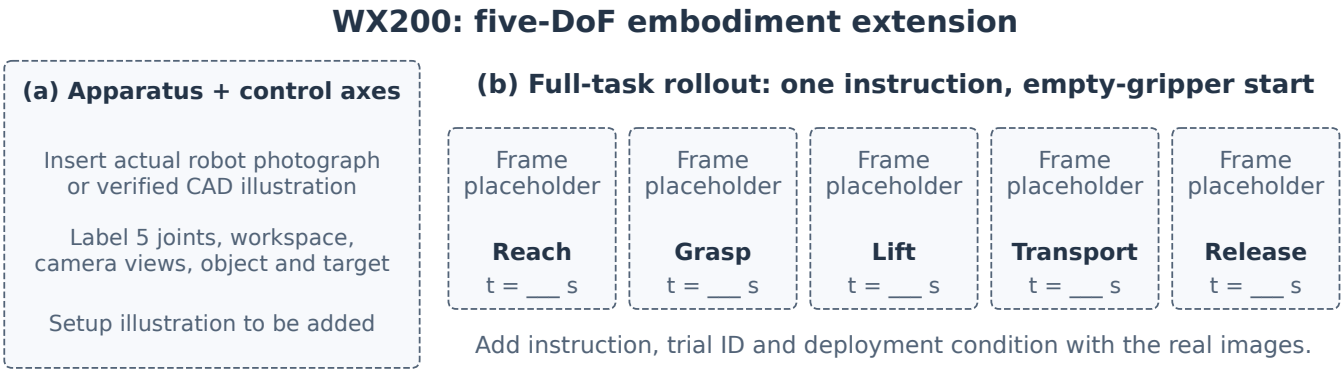


Fig. 6. WX200 evaluation layout. (a) Apparatus, workspace, cameras, and control axes. (b) Reach, grasp, lift, transport, and release under one instruction. The current panels reserve space for setup images and timestamped rollout frames from the ongoing experiment.

[5] H. Li, Y. Zuo, J. Yu, Y. Zhang, Z. Yang, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui, D. Wang, D. Luo, Y. Fan, Y. Sun, J. Zeng, J. Pang, S. Zhang, Y. Wang, Y. Mu, B. Zhou, and N. Ding, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," *arXiv preprint arXiv:2509.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2509.09674

[6] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: https://arxiv.org/abs/2402.03300

[7] Y. Mroueh, "Reinforcement learning with verifiable rewards: GRPO's loss, dynamics, and success amplification," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 90 821–90 847. [Online]. Available: https://proceedings.mlr.press/v306/mroueh26a.html

[8] Y. Y. Bay and K. A. Yearick, "GRPO, Dr. GRPO, and DAPO are three operations on one number: The group-standard-deviation identity," *arXiv preprint arXiv:2607.00152*, 2026. [Online]. Available: https://arxiv.org/abs/2607.00152

[9] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning*, 1999, pp. 278–287. [Online]. Available: https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf

[10] Y. Fujita and S.-i. Maeda, "Clipped action policy gradient," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1597–1606. [Online]. Available: https://proceedings.mlr.press/v80/fujita18a.html

[11] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.03310